

PROJECT DOCUMENTATION

AI FAQ ASSISTANT API

PROJECT TITLE- Artificial Intelligence - Frequently Asked Questions
Application Programming Interface

TEAM MEMBERS:

BHAVASRI R

BANU E

BHAVANI M

CHANDRA K

CHARULATHA N

INDEX

S.NO	TOPICS	PAGE NO
1.	Introduction	1
2.	Acknowledgment	2
3.	Problem Statement and Proposed Solution	3
4.	Objectives of the Project	4
5.	Software and Hardware Requirements	5
6.	Project Architecture	6
7.	ER Diagram Explanation	8
8.	Key Features	9
9.	User Flow	10
10.	MVC Architecture Pattern	13
11.	Project Setup and Configuration	14
12.	Project Working and Execution Process	15
13.	Features – AI FAQ Assistant API	18
14.	Advantages – AI FAQ Assistant API	19
15.	Disadvantages – AI FAQ Assistant API	20
16.	Conclusion	21

1.INTRODUCTION

The AI Assistant FAQ API is an intelligent application programming interface designed to provide automated answers to frequently asked questions. It uses Artificial Intelligence (AI) and Natural Language Processing (NLP) to understand user questions and generate relevant, accurate, and easy-to-understand responses.

Instead of manually searching through a list of FAQs, users can simply type their questions in natural language, and the AI assistant can identify the appropriate information and respond instantly. The API acts as a communication layer between the user application and the AI-powered FAQ system.

The AI Assistant FAQ API can be integrated into websites, mobile applications, customer-support platforms, educational systems, and chat bots. It can help organizations reduce repetitive support tasks, provide faster responses, and improve the overall user experience.

The system can use a predefined FAQ database or knowledge base to ensure that responses are based on available information. Depending on the application, the API can also support features such as question classification, keyword and semantic search, contextual responses, and continuous improvement of the knowledge base.

The basic working process begins when a user submits a question through an application. The question is sent to the API, where it is processed and analyzed using AI techniques.

The system searches the relevant FAQ or knowledge base, identifies the most appropriate information, and generates a suitable response. The answer is then returned to the application and displayed to the user.

2.ACKNOWLEDGMENT

I would like to express my sincere gratitude to everyone who contributed to the successful completion of my project titled “AI Assistant FAQ API.”

This project has been an important learning experience that helped me gain practical knowledge about Artificial Intelligence, APIs, natural language processing, database management, and software development.

First and foremost, I would like to thank my project guide for providing valuable guidance, continuous support, useful suggestions, and encouragement throughout the development of this project.

Their guidance helped me understand the concepts clearly and overcome various challenges during the implementation of the system.

I would also like to express my gratitude to the Head of the Department and all the faculty members of my department for providing the necessary knowledge, resources, and support required to complete this project successfully.

Their teaching and encouragement have played an important role in improving my technical skills and understanding of modern technologies.

I am thankful to my friends and classmates for their cooperation, suggestions, and support during the planning, development, testing, and documentation of the project. Their feedback helped me identify and improve several aspects of the application.

I would also like to thank my family for their constant encouragement, motivation, patience, and support throughout this project.

Their support gave me the confidence to complete the work successfully.

Finally, I thank everyone who directly or indirectly contributed to the successful completion of the AI Assistant FAQ API project.

This project has provided me with valuable practical experience and has strengthened my interest in developing intelligent and user-friendly software applications.

3. PROBLEM STATEMENT AND PROPOSED SOLUTION

Problem Statement

In today's digital environment, users frequently need quick answers to common questions related to products, services, applications, and general information.

Traditional FAQ systems usually provide a fixed list of questions and answers, requiring users to search manually for the information they need.

If the user's question is phrased differently from the available FAQ, finding the correct answer can become difficult and time-consuming.

At the same time, handling repetitive queries manually increases the workload of customer-support teams and may result in delays in providing assistance.

Therefore, there is a need for an intelligent and automated FAQ system that can understand user questions in natural language and provide relevant answers quickly.

The system should be easy to integrate with different applications and should reduce repetitive manual support while improving the overall user experience.

Proposed Solution

The proposed AI Assistant FAQ API provides an intelligent solution for automating frequently asked questions.

The system allows users to submit questions through an application, while the API processes the queries and identifies the most relevant answer from a predefined FAQ database or knowledge base.

Artificial Intelligence and Natural Language Processing techniques can be used to understand the meaning of user queries, even when they are expressed in different ways.

The API follows a simple process:

User Query → API Request → Query Processing → FAQ/Knowledge Base Search → Relevant Answer → API Response.

The system can be integrated with websites, mobile applications, chat bots, and customer-support platforms. It can provide quick and consistent responses, reduce the workload of support staff, and improve accessibility to information.

The FAQ database can also be updated whenever new questions and answers are added. This makes the system flexible and scalable according to the requirements of an organization.

4. OBJECTIVES OF THE PROJECT

- ❖ To implement secure communication between the client application and the API.
- ❖ To validate user inputs before processing the submitted queries.
- ❖ To handle invalid, incomplete, or unsupported questions appropriately.
- ❖ To provide meaningful error messages when a suitable answer cannot be found.
- ❖ To maintain a standardized response format for easy API consumption.
- ❖ To improve the organization and accessibility of FAQ information.
- ❖ To design the system with separate components for query processing, data retrieval, and response generation.
- ❖ To maintain consistency in the responses provided to similar queries.
- ❖ To monitor API performance and identify possible areas for improvement.
- ❖ To ensure that the system can be maintained and modified easily by developers.
- ❖ To provide a foundation for adding multilingual question-answering capabilities in the future.
- ❖ To support future integration with authentication, analytic, and administrative features.
- ❖ To provide a clear interface between the AI service and external applications.
- ❖ To reduce dependency on manually maintained support interfaces

5. SOFTWARE AND HARDWARE REQUIREMENTS

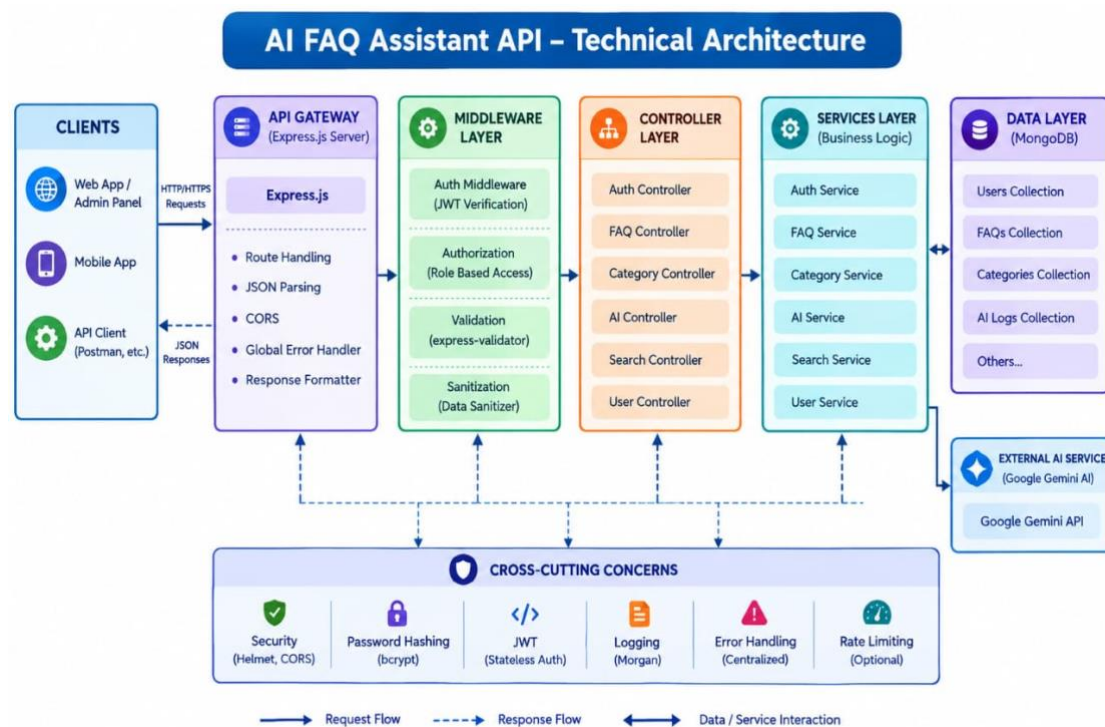
Software Requirements:

- ❖ Operating System: Windows 10/11
- ❖ Programming Language: Python 3.10 or above
- ❖ Backend: FastAPI
- ❖ API Server: Uvicorn
- ❖ Database: MySQL / SQLite
- ❖ Frontend: HTML, CSS, JavaScript
- ❖ Code Editor: Visual Studio Code
- ❖ Browser : Google Chrome / Microsoft Edge
- ❖ API Testing: Postman
- ❖ AI API: Open AI API or another suitable AI API
- ❖ Python Libraires: fastapi, uvicorn, requests, python-dotenv

Hardware Requirements:

- ❖ Processor: Intel Core i3 or above
- ❖ RAM: 4 GB minimum, 8 GB recommended
- ❖ Storage: 10 GB free space
- ❖ Internet: Stable internet connection
- ❖ GPU: Not required if using a cloud AI API

6. PROJECT ARCHITECTURE



The AI FAQ Assistant application exposes structured endpoints handled by a layered server architecture model. Incoming requests undergo filter processing before interacting with data services.

MAJOR COMPONENTS:

1.Clients

Clients are the applications that interact with the AI FAQ Assistant API. They include the web application, admin panel, mobile application, and API testing tools such as Postman.

2.API Gateway

The API Gateway is the entry point for client requests. It handles routing, JSON parsing, CORS, response formatting, and global error handling using **Express.js**.

3.Middleware Layer

This layer performs common processing before requests reach the controllers. It includes **JWT authentication, role-based authorization, input validation, and data sanitization**.

4.Controller Layer

Controllers receive API requests and coordinate the required operations. The system contains **Auth, FAQ, Category, AI, Search, and User Controllers**.

5.Services Layer

The Services Layer contains the main business logic of the application. It includes **Auth Service, FAQ Service, Category Service, AI Service, Search Service, and User Service**.

6.Data Layer

The Data Layer manages persistent application data using **MongoDB**. It contains collections for **Users, FAQs, Categories, and AI Logs**, along with other required data.

7.External AI Service

The External AI Service connects the application with **Google Gemini AI**. It processes user questions and generates intelligent responses when an AI-based answer is required.

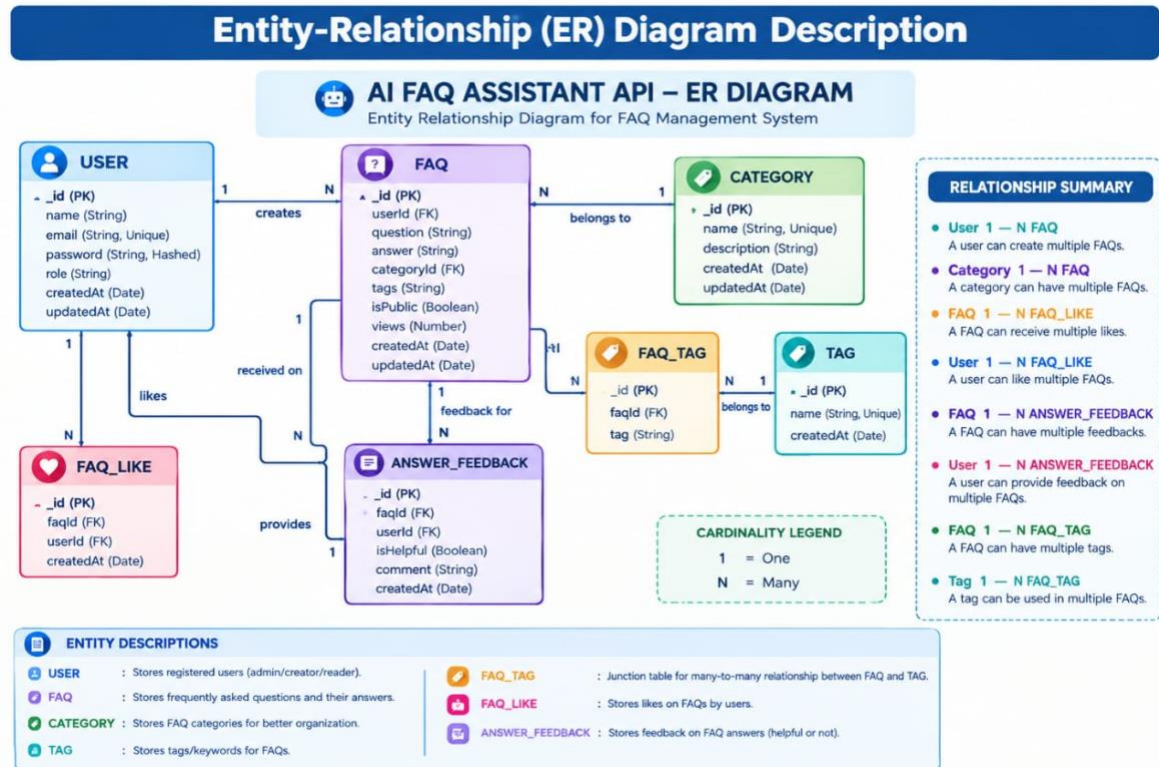
8.Cross-Cutting Concerns

These are common functions used throughout the system, including **security, password hashing, JWT authentication, logging, centralized error handling, and optional rate limiting**.

9.Request and Response Flow

A client sends an HTTP/HTTPS request through the API Gateway. The request passes through middleware and controllers to the service layer, which interacts with the database or external AI service. The processed result is then returned to the client as a JSON response.

7. ER DIAGRAM EXPLANATION



The **Entity-Relationship (ER) diagram** represents the database structure of the AI FAQ Assistant API. It shows the main entities, their attributes, and the relationships between users, FAQs, categories, tags, likes, and feedback.

1. USER

The **USER** entity stores registered user information such as user ID, name, email, password, role, and account creation and update dates. A user can create multiple FAQs, like multiple FAQs, and provide feedback on multiple FAQ answers.

2. FAQ

The **FAQ** entity stores frequently asked questions and their answers. It contains the question, answer, user ID, category ID, tags, visibility status, number of views, and timestamps. Each FAQ is created by one user and belongs to one category.

3. CATEGORY

The **CATEGORY** entity organizes FAQs into different categories. It stores the category name, description, and timestamps. One category can contain multiple FAQs, while each FAQ belongs to one category.

4. TAG and FAQ_TAG

The **TAG** entity stores keywords used to classify FAQs. Since one FAQ can have multiple tags and one tag can be associated with multiple FAQs, the **FAQ_TAG** entity acts as a junction table to manage this many-to-many relationship.

5. FAQ_LIKE

The **FAQ_LIKE** entity records likes given by users to FAQs. It connects a user with an FAQ, allowing a user to like multiple FAQs and an FAQ to receive likes from multiple users.

6. ANSWER_FEEDBACK

The **ANSWER_FEEDBACK** entity stores user feedback about FAQ answers. It records whether an answer was helpful, along with optional comments and timestamps. One FAQ can receive multiple feedback entries.

7. Relationships

The main relationships are:

- ❖ **User 1 → N FAQ:** One user can create many FAQs.
- ❖ **Category 1 → N FAQ:** One category can contain many FAQs.
- ❖ **FAQ 1 → N FAQ_LIKE:** One FAQ can receive many likes.
- ❖ **User 1 → N FAQ_LIKE:** One user can like many FAQs.
- ❖ **FAQ 1 → N ANSWER_FEEDBACK:** One FAQ can receive multiple feedback entries.
- ❖ **User 1 → N ANSWER_FEEDBACK:** One user can provide feedback on multiple FAQs.
- ❖ **FAQ 1 → N FAQ_TAG:** An FAQ can have multiple tags.
- ❖ **Tag 1 → N FAQ_TAG:** A tag can be associated with multiple FAQs.

8. Cardinality

The diagram uses **1** to represent one and **N** to represent many. These cardinalities clearly define how records in one entity are connected to records in another entity.

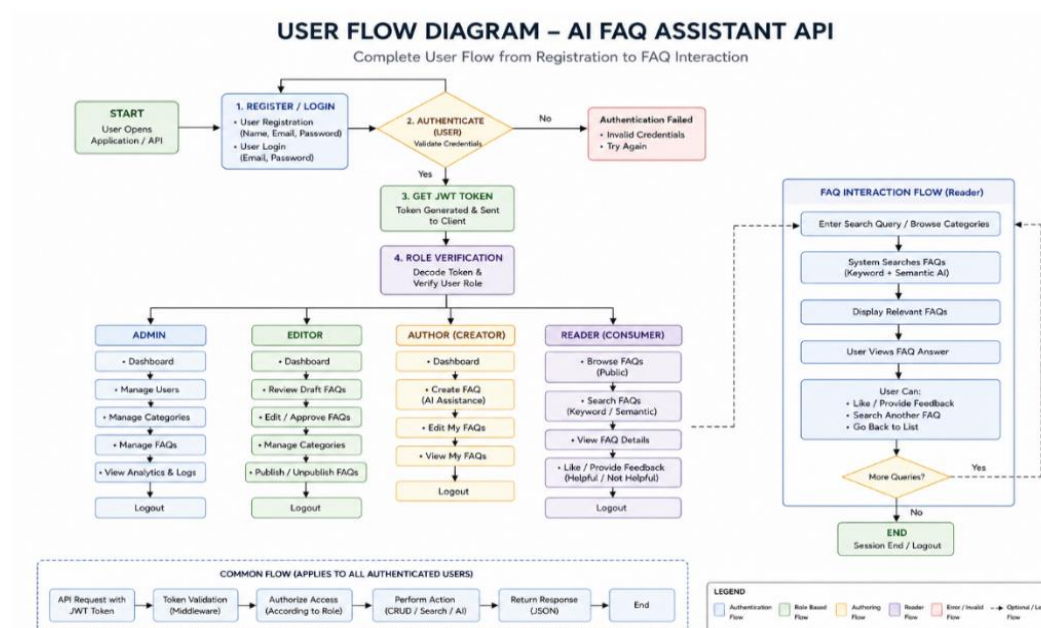
8.Key Features

- ❖ **JWT Authentication & Authorization:** Secure token-based authentication protects private API endpoints.

- ❖ **AI FAQ Generation:** Google Gemini AI automatically generates questions, answers, and topic suggestions.
- ❖ **Semantic FAQ Search:** Intelligent keyword-based search enables faster information retrieval.
- ❖ **Complete CRUD Operations:** Create, Read, Update, and Delete FAQ records.
- ❖ **Category Management:** Organizes FAQs into structured categories for easier navigation.
- ❖ **Password Encryption:** User passwords are securely hashed using bcrypt before storage.
- ❖ **Centralized Error Handling:** Standardized API error responses improve reliability and debugging.
- ❖ **Schema Validation:** Mongoose validation ensures data integrity before database persistence.
- ❖ **RESTful API Architecture:** Modular Express.js routes provide scalable backend services.
- ❖ **MongoDB Integration:** Efficient NoSQL document storage using Mongoose ODM.

9. USER FLOW

User Flow



The **User Flow Diagram** explains how different users interact with the AI FAQ Assistant API, starting from registration/login and continuing through authentication, role verification, FAQ interaction, and logout.

1. Start

The process begins when a user opens the **web application, mobile application, or API client**.

2. Register / Login

The user can register a new account or log in using their **email/username and password**.

3. Authentication

The system verifies the submitted credentials.

- ❖ If the credentials are **incorrect**, authentication fails and the user is asked to try again.
- ❖ If the credentials are **valid**, the user proceeds to the next step.

4. JWT Token Generation

After successful authentication, the system generates a **JWT (JSON Web Token)**. The token is sent to the client and is used for securely accessing protected API resources.

5. Role Verification

The system verifies the user's role from the authenticated token. Based on the role, the user receives different permissions.

6. Role-Based User Flow

Admin

- ❖ Access dashboard
- ❖ Manage users
- ❖ Manage categories
- ❖ Manage FAQs
- ❖ View analytics and logs
- ❖ Logout

Editor

- ❖ Access dashboard
- ❖ Create/edit FAQs
- ❖ Edit or approve FAQ content
- ❖ Manage categories

- ❖ Publish or update FAQs
- ❖ Logout

Author / Creator

- ❖ View dashboard
- ❖ Create FAQs using AI assistance
- ❖ Edit FAQs
- ❖ View personal FAQs
- ❖ Logout

Reader / Consumer

- ❖ Browse public FAQs
- ❖ Search FAQs using keywords
- ❖ View FAQ details
- ❖ Like FAQs
- ❖ Provide feedback
- ❖ Logout

7. FAQ Interaction Flow

For users accessing FAQs, the system provides a common interaction process:

Enter Search Query / Select Category → System Searches FAQs → Display Relevant FAQs → User Views Answer → User Can Like / Provide Feedback

If an appropriate answer is not available, the system can use the **AI service** to generate or assist with an answer.

8. Common API Flow

All authenticated requests follow a common process:

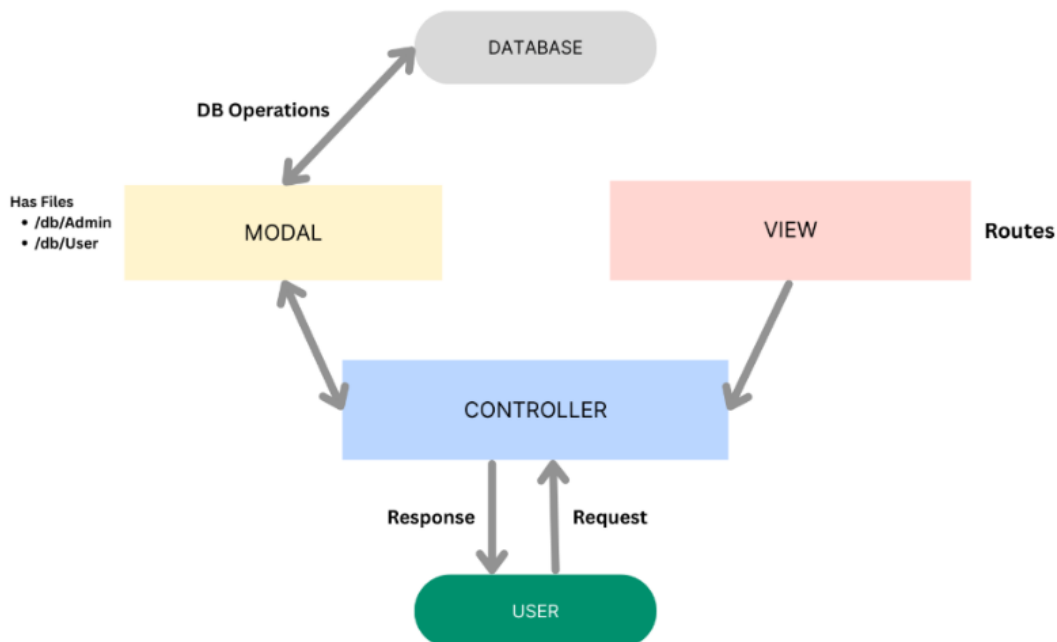
API Request with JWT → Token Validation → Access Verification According to Role → Perform Action (CRUD / Search / AI) → Return JSON Response → End

9. Logout

After completing their activities, users can log out. This ends the current authenticated session and prevents further protected operations until the user authenticates again.

10. MVC ARCHITECTURE PATTERN

The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:



The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:

Model Layer (Data Blueprint)

Defines structural blueprints via strongly enforced Mongoose schemas mapping explicit fields into raw collection items inside MongoDB. It sets field rules and defaults before validation persistence occurs.

Controller Layer (Execution Matrix)

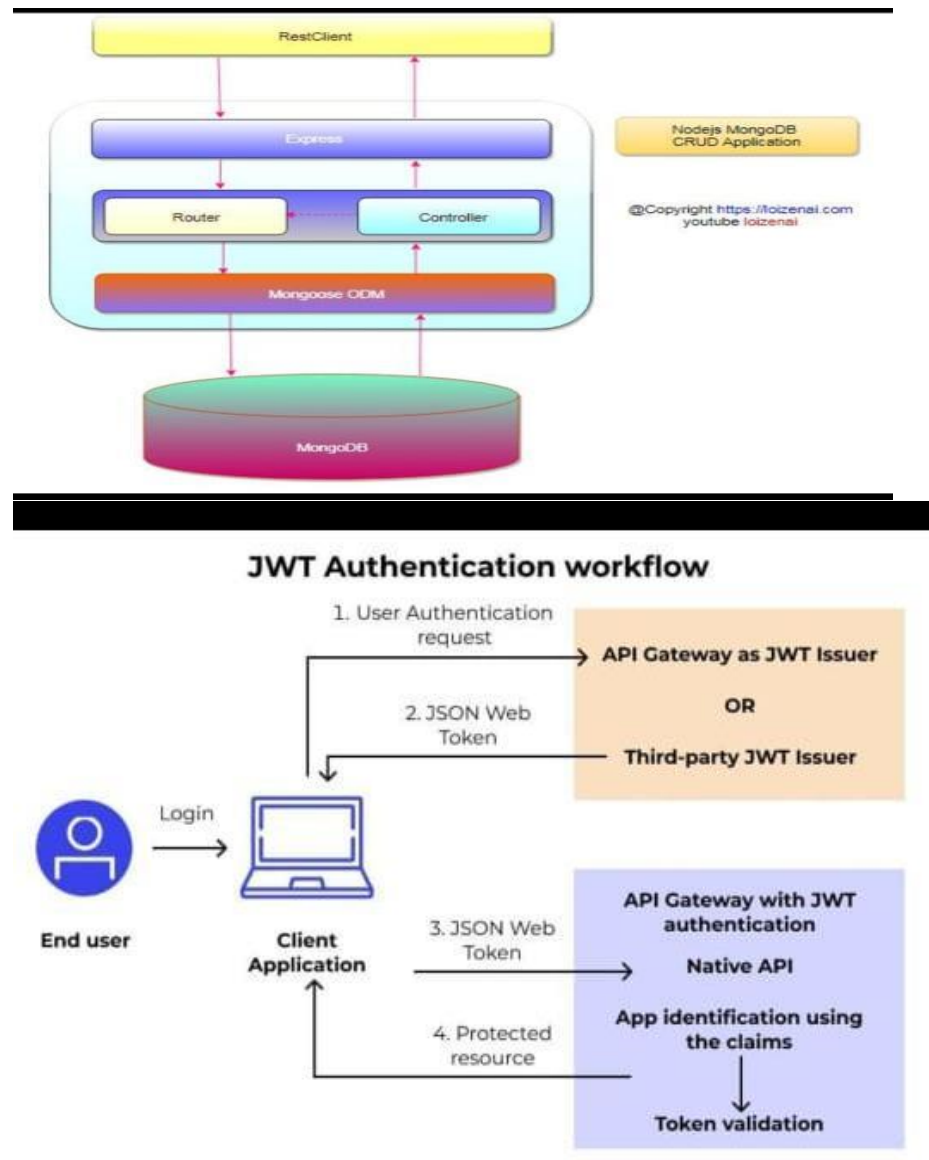
The intermediary brain of the ecosystem. It captures request vectors from execution routes, checks structural request constraints, delegates processing workflows directly to models or services, and packages resultant output into standard JSON return arrays.

View Layer (Routing Network Interface)

In this pure headless backend service context, standard user-interface template views do not exist. Instead, the interface layer lives as an API Routing Layer. It links network requests on specified paths directly to their associated controller handler logic maps.

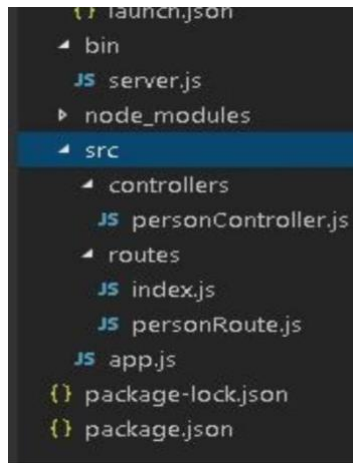
11. PROJECT SETUP AND CONFIGURATION

The Project Configuration defines the tools, technologies, database, security mechanisms, and environment settings required to develop and run the AI FAQ Assistant API.



- ❖ **Development Environment** – Windows/Linux/macOS, Visual Studio Code, Node.js, npm, and Postman.
- ❖ **Backend Configuration** – Express.js with JavaScript/Node.js, REST API architecture, and JSON data format.
- ❖ **Database Configuration** – MongoDB with Mongoose for storing users, FAQs, categories, tags, likes, feedback, and AI logs.
- ❖ **Security Configuration** – JWT authentication, bcrypt password hashing, Helmet, CORS, input validation, sanitization, and role-based access control.

- ❖ **AI Configuration** – Google Gemini AI is integrated to provide AI-assisted answers when required.
- ❖ **Environment Configuration** – Database URL, JWT secret, API keys, and server settings are stored securely in the .env file.
- ❖ **Project Structure** – The project is organized into controllers, middleware, models, routes, services, configuration, utilities, logs, and the main server.js file.

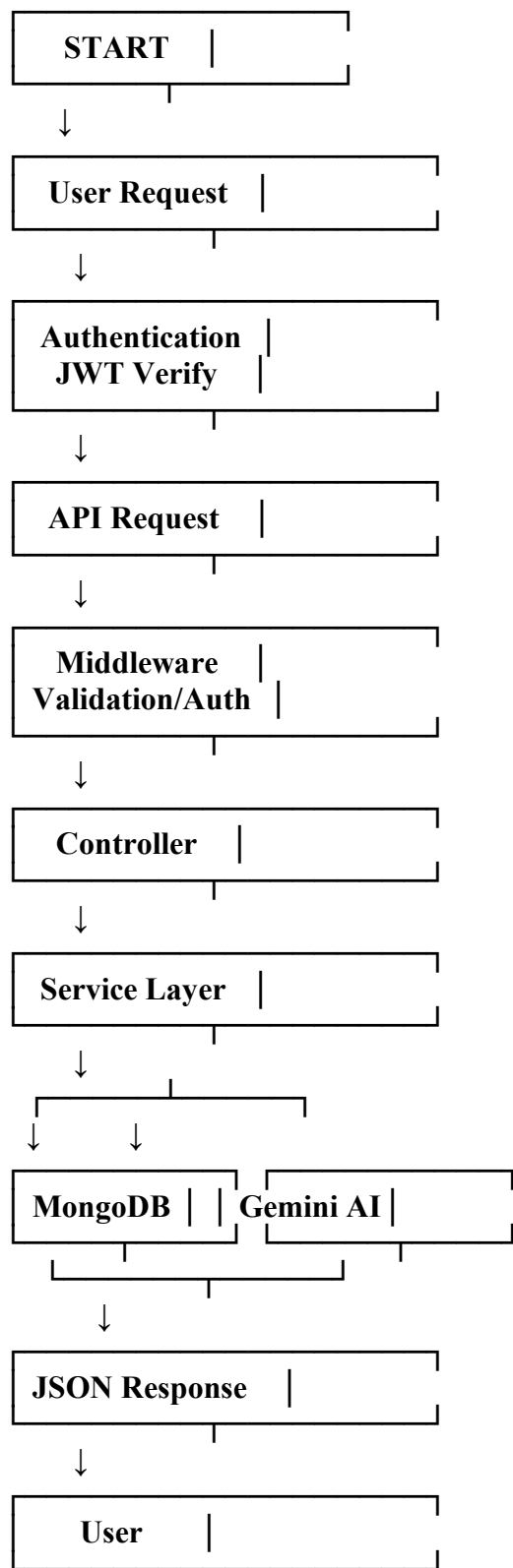


12. PROJECT WORKING AND EXECUTION PROCESS

WORKING PROCESS

- ❖ **Development Environment** – Windows/Linux/macOS, Visual Studio Code, Node.js, npm, and Postman.
- ❖ **Backend Configuration** – Express.js with JavaScript/Node.js, REST API architecture, and JSON data format.
- ❖ **Database Configuration** – MongoDB with Mongoose for storing users, FAQs, categories, tags, likes, feedback, and AI logs.
- ❖ **Security Configuration** – JWT authentication, bcrypt password hashing, Helmet, CORS, input validation, sanitization, and role-based access control.
- ❖ **AI Configuration** – Google Gemini AI is integrated to provide AI-assisted answers when required.
- ❖ **Environment Configuration** – Database URL, JWT secret, API keys, and server settings are stored securely in the .env file.
- ❖ **Project Structure** – The project is organized into controllers, middleware, models, routes, services, configuration, utilities, logs, and the main server.js file.

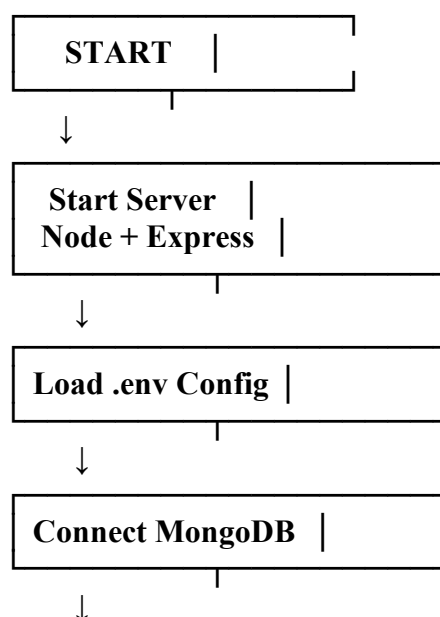
WORKING PROCESS-FLOWCHART

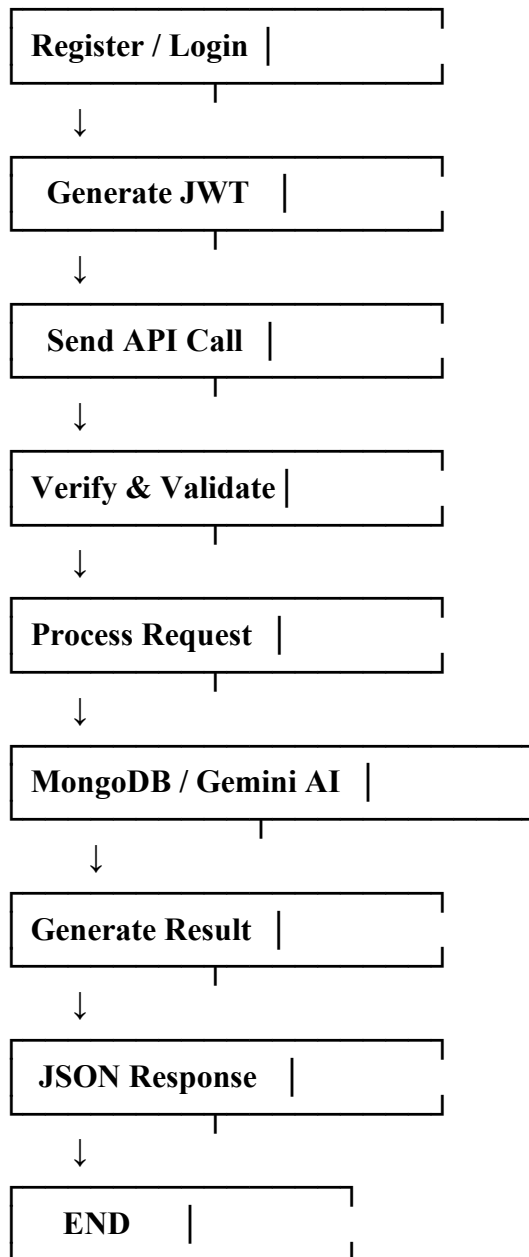


EXECUTION PROCESS

- ❖ **Start the Server** – Run the Node.js/Express.js application.
- ❖ **Connect Database** – Establish a connection with MongoDB
- ❖ **Load Configuration** – Read .env settings such as port, database URL, JWT secret, and Gemini API key.
- ❖ **User Authentication** – User registers or logs in and receives a JWT token.
- ❖ **Send Request** – The user sends an FAQ search, CRUD operation, or question through the API.
- ❖ **Verify Request** – Middleware validates the JWT token, user role, and input data.
- ❖ **Process Request** – The controller sends the request to the appropriate service.
- ❖ **Access Data / AI** – The service retrieves data from MongoDB or sends the question to Gemini AI when AI assistance is required.
- ❖ **Generate Response** – The system prepares the requested result.
- ❖ **Return Response** – The API sends the result to the client in JSON format.
- ❖ **Complete Operation** – The user views the answer or performs further actions such as updating, deleting, liking, or providing feedback.

EXECUTION PROCESS -FLOW DIAGRAM





13. Features – AI FAQ Assistant API

- ❖ **User Registration and Login** – Allows users to securely create accounts and log in.
- ❖ **JWT Authentication** – Protects API endpoints using JSON Web Tokens.
- ❖ **Role-Based Access Control** – Provides different permissions for Admin, Editor, Author, and Reader users.
- ❖ **FAQ Management** – Users with appropriate permissions can create, view, update, and delete FAQs.

- ❖ **FAQ Search** – Allows users to quickly search FAQs using keywords.
- ❖ **Category Management** – Organizes FAQs into different categories for easy access.
- ❖ **Tag Management** – Uses tags to classify and find related FAQs.
- ❖ **AI-Powered Answers** – Integrates Google Gemini AI to provide intelligent answers when required.
- ❖ **Like and Feedback System** – Users can like FAQs and provide feedback about answers.
- ❖ **Secure Password Storage** – Uses password hashing to protect user credentials.
- ❖ **Input Validation** – Validates user input before processing API requests.
- ❖ **API Error Handling** – Provides structured error responses when requests fail.
- ❖ **Logging** – Records important API and system activities for monitoring and debugging.
- ❖ **RESTful API** – Provides structured endpoints for communication between the client and backend.
- ❖ **JSON Response** – Returns API results in a standard JSON format.

14. ADVANTAGES - AI FAQ ASSISTANT API

- ❖ **Fast Response** – Provides quick answers to frequently asked questions.
- ❖ **AI-Powered Assistance** – Uses AI to generate useful answers for questions not covered by existing FAQs.
- ❖ **Easy FAQ Management** – Supports creating, viewing, updating, and deleting FAQ content.
- ❖ **Improved User Experience** – Makes information easier to search and access.
- ❖ **Secure Authentication** – Protects user accounts using JWT authentication and password hashing.
- ❖ **Role-Based Access** – Provides different access permissions for different types of users.
- ❖ **Centralized Data Storage** – Stores FAQ and user information in MongoDB.

- ❖ **Better Organization** – Categories and tags make FAQs easier to organize and find.
- ❖ **Scalable Architecture** – The modular API structure makes it easier to add new features.
- ❖ **User Feedback** – Likes and feedback help identify useful FAQ answers.
- ❖ **Reduced Manual Work** – Automates repetitive FAQ answering and management tasks.
- ❖ **Easy Integration** – REST APIs can be integrated with web, mobile, or other client applications.

15. DISADVANTAGES -AI FAQ ASSISTANT API

- ❖ **Internet Dependency** – The AI service and API may require a stable internet connection.
- ❖ **AI Response Accuracy** – AI-generated answers may sometimes be incorrect or incomplete.
- ❖ **API Cost** – Using external AI services such as Gemini may involve usage limits or costs.
- ❖ **Database Dependency** – System performance depends on proper MongoDB connectivity and availability.
- ❖ **Security Risks** – Improper handling of user data, API keys, or authentication tokens can create security vulnerabilities.
- ❖ **Maintenance Required** – FAQs, categories, tags, and AI configurations need regular maintenance.
- ❖ **Response Delay** – AI-generated responses may take longer than retrieving an answer directly from the database.
- ❖ **Complex Development** – Integrating authentication, database operations, APIs, and AI services increases development complexity.
- ❖ **Rate Limits** – External AI APIs may have request limits that can affect availability during heavy usage.
- ❖ **Data Privacy Concerns** – Sensitive information should not be unnecessarily sent to external AI services.

16. CONCLUSION

The AI FAQ Assistant API is designed to provide a simple, efficient, and intelligent solution for handling frequently asked questions. The main purpose of this project is to reduce the effort required to answer repeated user queries and to provide relevant information in a faster and more organized manner. By integrating Artificial Intelligence with an Application Programming Interface (API), the system can receive questions from users, process the queries, identify the most relevant information, and return suitable responses. This makes the system useful for applications where users need quick access to frequently requested information.

The project demonstrates the practical use of AI technologies in developing modern and user-friendly applications. Instead of depending completely on manual support, the FAQ assistant can automatically respond to commonly asked questions, thereby reducing response time and improving accessibility. The API-based design also allows the FAQ assistant to communicate with different client applications, websites, or other software systems. This provides flexibility and makes the system easier to integrate into different platforms.

Another important advantage of the system is its ability to organize FAQ information in a structured manner. Users can interact with the system by submitting questions without needing to search through large amounts of information manually. The system can provide consistent responses for commonly available questions and can be updated whenever new information or frequently asked questions are added. Proper validation and error handling can also improve the reliability of the application and ensure that users receive meaningful responses.

The project also provides a strong foundation for future development. The system can be enhanced by adding a larger knowledge base, improved natural language processing, multilingual support, voice-based interaction, authentication, analytics, and better response generation. It can also be integrated with databases and other external services to provide more dynamic information. Continuous improvement of the FAQ data and AI model can help the system handle a wider range of user queries effectively.

In conclusion, the AI FAQ Assistant API demonstrates how Artificial Intelligence and API technology can be combined to create an automated question-answering system. It improves the way frequently requested information is delivered while reducing repetitive manual work. The project provides a practical and scalable foundation that can be extended with additional features in the future. Thus, the AI FAQ Assistant API can serve as a useful component for developing intelligent, responsive, and efficient applications.